Level Two

6-Month GenAI Engineering Course

AI Foundations · FastAPI · LangChain · RAG · MCP · Multi-Agent · Docker

Total Duration: 6 Months (24 Weeks) | Classes per Week: 3 | Total Classes: 72

Prerequisite: Level One — Python, AI & Automation Course

End Goal

Students who complete Level One will build on their Python, ML, and GenAI foundations to create production-ready AI-powered web applications. By the end of Level Two, students will be able to:

- Explain the AI hierarchy — AI, ML, Deep Learning, Neural Networks, Transformers, LLMs, GenAI — and how they connect
- Understand how LLMs actually work: tokenization, embeddings, attention, hallucinations, and grounding
- Design and build REST APIs using FastAPI — the industry-standard Python web framework
- Use LangChain to build AI applications with chains, memory, tools, and agents
- Implement Retrieval-Augmented Generation (RAG) pipelines with vector databases to eliminate hallucinations
- Understand and apply the Model Context Protocol (MCP) for standardized AI tool use
- Build multi-agent systems using LangGraph where specialized agents collaborate to solve complex tasks
- Create user interfaces with Streamlit and AI-generated frontends using Lovable.dev
- Containerize applications using Docker and deploy them to the cloud
- Use Claude Code (free via OpenRouter + ccr) as an AI-powered development assistant throughout the course

🤖 **Claude Code — Built Into the Course**

One Claude Code class is embedded as Class 3 of every alternate week (Weeks 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24) — 12 classes in total. No additional sessions are added; these replace the standard Class 3 of that week.

Free Setup: Claude Code CLI + claude-code-router (ccr) + OpenRouter free models (Qwen3-Coder, Gemini Flash). Full setup guide available in the Panaversity AI Agent Factory documentation.

Phase-wise Structure (6 Months)

Phase	Duration	Weeks	Focus	Main Outcome
Phase 1: AI Foundations + FastAPI	5 weeks	Weeks 1–5	AI/ML/DL/LLM concepts → FastAPI core → Auth, DB → GenAI backend	Students understand how AI works and can build LLM-powered APIs
Phase 2: LangChain Core	7 weeks	Weeks 6–12	Prompts, LCEL, Memory, Tools, Agents, LangSmith	Build production-grade AI chatbots with tool calling
Phase 3: RAG + MCP + Streamlit	5 weeks	Weeks 13–17	Embeddings, ChromaDB, RAG pipelines, MCP, Streamlit UI	Build and deploy a full document Q&A application
Phase 4: Multi-Agent + LangGraph	3 weeks	Weeks 18–20	OpenAI Agents SDK concepts, LangGraph, Supervisor agents	Design multi-agent workflows for real-world use cases
Phase 5: Docker + Deployment + Capstone	4 weeks	Weeks 21–24	Docker, Compose, Deployment, Lovable frontend, Capstone	Ship a complete production-ready GenAI application

🔵 Phase 1: AI Foundations + FastAPI (Weeks 1–5)

Duration: 5 weeks · 15 classes | Focus: Week 1: AI/ML/DL/LLM/GenAI foundations → Weeks 2–5: FastAPI core, Auth, DB, GenAI backend

Outcome: Students understand how AI works under the hood and can build their own LLM-powered REST APIs

Phase 1 has two distinct parts. Week 1 is a dedicated AI Foundations week — students learn the entire AI hierarchy from ML to GenAI, understand how LLMs actually work (tokenization, embeddings, attention, hallucinations), and get context for everything they will build in the next 23 weeks. This is not revision of Level One — it goes deeper into the concepts behind the tools. Weeks 2 to 5 then move into FastAPI, where students flip from API consumers (Level One) to API builders.

Week 1 — AI Foundations (Dedicated Foundation Week)

Why a dedicated foundations week?

Level One gave students scikit-learn and basic ML hands-on. But Level Two builds full GenAI applications — and students need to understand WHY things like RAG, embeddings, vector databases, and hallucinations exist before they build them.

This week answers: How does an LLM actually work? What is a Transformer? Why do LLMs hallucinate? What is an embedding? Why does RAG solve the context-window problem? These answers make every subsequent topic click into place.

Reference resources: Google Cloud AI & ML documentation, deeplearning.ai GenAI with LLMs course structure, and the Panaversity learn-agentic-ai repository foundations.

Week	Class 1	Class 2	Class 3
Week 1	AI landscape overview: AI → ML → Deep Learning → Neural Networks → Transformers → LLMs → GenAI. Visual hierarchy. Level One ML recap — where scikit-learn fits in. Real-world examples of each layer.	How LLMs actually work: Tokenization (hands-on demo), context window concept, attention mechanism (intuition only — no math), temperature setting, hallucinations — what they are and why they happen. AI hallucinations discussion with real examples.	Embeddings & semantic meaning: What embeddings are (numbers that represent meaning). Why similar meanings are close in vector space. Live demo — compare sentence similarities. This is the foundation of RAG — preview of Week 13.

AI Hierarchy Reference Table

The table below is designed for classroom use in Class 1 of Week 1 — to be built together with students progressively during the session:

Concept	What it is	Real-world example	Relevance in this course
Artificial Intelligence (AI)	Umbrella term — machines that perform tasks requiring human intelligence	Face recognition, spam filter, Google Maps	Everything we build is AI
Machine Learning (ML)	Systems that learn patterns from data without explicit programming	Netflix recommendations, fraud detection	Level 1 foundation — Linear Regression, Classification
Deep Learning (DL)	ML using multi-layer neural networks — learns complex patterns automatically	Image recognition, speech-to-text, ChatGPT	Powers the LLMs we use via APIs
Neural Networks	Layers of connected nodes that learn representations from data	A brain-inspired structure — input → hidden layers → output	Core architecture behind every LLM
Transformers	Neural network architecture using attention — what all modern LLMs are built on	GPT, Claude, Gemini, LLaMA are all Transformer models	Why LLMs understand context so well
Large Language Models (LLMs)	Transformers trained on massive text data — generate and understand language	ChatGPT, Claude, Groq-hosted Llama	Primary tool in this course — via API
Generative AI (GenAI)	AI that creates new content (text, images, code) rather than just classifying	ChatGPT writing code, DALL-E making images	The application layer we build on top of LLMs

Key LLM Concepts Reference Table

The table below covers Class 2 of Week 1 — core concepts students need before touching any LangChain or RAG code:

Concept	Simple explanation	Why it matters for building apps
Tokenization	Text is broken into small pieces called tokens before the LLM processes it	Explains why LLMs have a context limit — tokens cost money and space
Context window	Maximum amount of text (tokens) an LLM can see at one time	Why RAG exists — you cannot fit a whole book in the context window

Embeddings	Numbers that represent the meaning of text in vector space — similar meanings are close together	The entire foundation of RAG — semantic search uses embeddings
Attention mechanism	How the model decides which words are most important when generating a response	Why LLMs understand relationships across long text (not just nearby words)
Temperature	A setting that controls how random or deterministic the output is	Low temp = predictable answers, high temp = more creative but less accurate
Hallucination	When an LLM confidently generates incorrect or fabricated information	Why we build RAG — to ground responses in real, retrieved facts
Grounding	Connecting LLM output to verified external data sources	Core purpose of RAG — grounded generation is accurate generation

Weeks 2–5 — FastAPI

Level One students used the requests library to consume APIs. In Weeks 2–5, they flip the perspective — they become API builders. FastAPI auto-generates Swagger documentation and handles validation out of the box. The mini project at the end of Week 5 ties FastAPI directly to a free LLM call using Groq — making immediate use of the LLM concepts from Week 1.

Week	Class 1	Class 2	Class 3
Week 2	Flask in 1 class — Hello World, routes, jsonify. Flask vs FastAPI comparison. FastAPI installation. First endpoint. Automatic Swagger UI — live demo.	Path parameters, query parameters. Multiple endpoints. Response models. POST, PUT, DELETE — CRUD basics.	🤖 Claude Code Setup: npm install ccr, OpenRouter free account, config.json, ccr start + ccr code. Basic commands and first Claude Code session.
Week 3	Pydantic models — request and response validation. Error handling, HTTP status codes (400, 422, 500). Exception handlers.	Dependency Injection basics. Middleware — logging, CORS. JWT authentication — theory and implementation.	SQLAlchemy + SQLite — models, sessions, CRUD with a real database.
Week 4	Background tasks. File uploads. .env secrets management. Project structure best practices.	Testing with pytest — basic endpoint tests. FastAPI + LLM integration: first GenAI API endpoint using the Groq free API (connects	🤖 Claude Code Class: Generate a FastAPI project scaffold using Claude Code. CLAUDE.md and SKILL.md concepts.

		directly to Week 1 LLM concepts).	Panaversity Agent Factory methodology intro.
Week 5	Mini Project (Part 1): Simple GenAI Backend API — FastAPI + Groq free LLM endpoint. Students choose their own use case.	Mini Project (Part 2): Polish, error handling, Swagger documentation clean-up.	Mini Project Presentations: Students demo via Swagger UI. Peer feedback session.

🟣 Phase 2: LangChain Core (Weeks 6–12)

Duration: 7 weeks · 21 classes | Focus: LangChain intro → Prompts, LCEL → Memory → Tools → Agents → LangSmith → Chatbot Project

Outcome: Students can build a production-grade AI chatbot with memory, tool calling, and a FastAPI backend

Level One students made raw LLM calls with Groq and OpenAI APIs. LangChain builds on top of those calls — adding structure, composability, and production patterns. This phase uses Groq's free API throughout. Agent concepts are first introduced using the OpenAI Agents SDK as a conceptual reference (Agent, Tool, Handoff, Guardrail, Runner — from the Panaversity learn-agentic-ai repository), then implemented in LangChain. The Week 1 foundations — especially tokenization, context windows, and hallucinations — will be directly referenced throughout this phase.

Week	Class 1	Class 2	Class 3
Week 6	Why LangChain? Problems with raw LLM calls. Installation. Groq free API setup. ChatModels. PromptTemplates — static and dynamic.	First chain: prompt | llm | parser. LCEL (LangChain Expression Language) — the pipe operator. Chain composition patterns.	🤖 Claude Code Class: Scaffold a LangChain project. File editing with Claude Code agent. /add, /clear commands.
Week 7	Output parsers — StrOutputParser, JsonOutputParser. Structured outputs. LangChain + FastAPI — first proper GenAI API endpoint.	Memory types — ConversationBufferMemory, ConversationSummaryMemory. How they connect to the context window concept from Week 1.	Chat history management. Multi-turn conversations. Session handling.
Week 8	Tools in LangChain — what they are and why they matter. @tool decorator. Custom tool building.	Built-in tools — DuckDuckGo search, Wikipedia. Tool descriptions and how the LLM decides which to use.	🤖 Claude Code Class: SKILL.md files — create one for your project. Claude Code handles multi-file edits.
Week 9	Tool calling with LLMs — bind_tools. Letting the model decide when to use a tool. ReAct pattern (Reason + Act).	Agents — conceptual intro using OpenAI Agents SDK: Agent, Tool, Handoff, Guardrail, Runner (Panaversity repo reference). Implement with LangChain: create_react_agent, AgentExecutor.	Agent with multiple tools — web search + custom tool. Observing agent reasoning steps.
Week 10	LangSmith — free tier setup. Tracing chains and	Token cost control. Evaluation basics.	🤖 Claude Code Class: Multi-file project workflow.

	agents. Debugging failed runs visually.	Production best practices. How hallucinations (Week 1) show up in traces and how to catch them.	/compact command. Subagents concept.
Week 11	Mini Project (Part 1): AI Chatbot — FastAPI backend + LangChain + Memory + Tools.	Mini Project (Part 2): Polish, error handling, edge cases. Code review session.	Mini Project Presentations: Students demo their chatbot. Live Q&A and peer feedback.
Week 12	Revision — common LangChain patterns, best practices review. Q&A on Phase 2 concepts.	Buffer / catch-up class. Students who need extra time on their chatbot project use this session.	🤖 Claude Code Class: Improve your chatbot using Claude Code. Portfolio GitHub setup and README generation.

🟢 Phase 3: RAG + MCP + Streamlit (Weeks 13–17)

Duration: 5 weeks · 15 classes | Focus: Embeddings deep dive → ChromaDB → RAG pipeline → Streamlit UI → MCP → Document Q&A Project

Outcome: Students can build and deploy a document Q&A application with a full Streamlit UI

This phase directly builds on Week 1 foundations. Embeddings were introduced conceptually in Week 1 Class 3 — here students implement them. Hallucinations were explained in Week 1 — here students build the solution (RAG). The context window limitation was covered in Week 1 — here students see exactly why chunking and retrieval exist. All tools are free: ChromaDB runs locally, Streamlit Cloud offers free deployment, and HuggingFace provides open-source embedding models.

Week	Class 1	Class 2	Class 3
Week 13	Embeddings revisited — from Week 1 concept to implementation. Semantic similarity hands-on. Free embedding models via HuggingFace sentence-transformers. Cosine similarity explained.	ChromaDB — local setup (completely free). Store documents as embeddings, query by semantic similarity. Why this solves the context window problem from Week 1.	What is RAG and why does it exist? Retrieval vs fine-tuning. The hallucination → grounding → RAG connection (Week 1 concepts applied).
Week 14	Document loaders — PDF, text files, web pages. Text splitters — chunk size and overlap. Why chunking matters for retrieval quality.	Full RAG pipeline: load → split → embed → store → retrieve → generate. LangChain implementation end-to-end.	🤖 Claude Code Class: Build the RAG project scaffold using Claude Code. Students generate their own document Q&A base.
Week 15	Streamlit intro — what it is and why it is the fastest way to prototype GenAI UIs. st.chat_message, st.chat_input, st.file_uploader.	RAG app in Streamlit — chat interface with file upload. Connect Streamlit frontend to FastAPI backend.	Streamlit Cloud deployment (free). Students publish and share their RAG app with a public link.
Week 16	MCP (Model Context Protocol) — what it is, why it was created, and how it standardizes tool calling across AI frameworks. Panaversity learn-agentic-ai repo reference.	LangChain MCP adapter — consuming tools from an MCP server inside a LangChain agent.	🤖 Claude Code Class: MCP servers in Claude Code — filesystem MCP, web MCP. Build a simple custom MCP server.
Week 17	Mini Project (Part 1): Smart Document Assistant — RAG pipeline	Mini Project (Part 2): Testing, edge cases, improvement. Upload multiple document types.	Project Presentations: Full demo with deployment link shared.

	+ MCP tool + Streamlit UI + FastAPI backend.		Instructor and peer feedback.

🟡 Phase 4: Multi-Agent Systems + LangGraph (Weeks 18–20)

Duration: 3 weeks · 9 classes | Focus: OpenAI Agents SDK concepts → LangGraph intro → Supervisor pattern → Multi-Agent Project

Outcome: Students can design and build multi-agent workflows for real-world use cases

With a solid LangChain foundation, students are ready to build systems where multiple AI agents collaborate. The OpenAI Agents SDK (from the Panaversity learn-agentic-ai repository, folder 01_ai_agents_first) is used conceptually to introduce key primitives — Agent, Handoff, Guardrail, Runner — before implementing the same patterns in LangGraph.

Week	Class 1	Class 2	Class 3
Week 18	Why multi-agent? Limitations of a single agent. OpenAI Agents SDK — conceptual walkthrough: Agent, Tool, Handoff, Guardrail, Runner (Panaversity reference, no implementation).	LangGraph intro — nodes, edges, state. Graph-based agent execution. First stateful agent.	🤖 Claude Code Class: Spec-first development from Panaversity AI Agent Factory. Write a specification before building. Claude Code generates from spec.
Week 19	Supervisor + Specialist agent pattern. Routing logic. Agent handoffs in LangGraph.	Real use cases: Research agent, Customer support bot, Code review agent. Architecture discussion.	Mini Project (Part 1): Multi-Agent App — supervisor agent + 2 to 3 specialist agents for a real use case.
Week 20	Mini Project (Part 2): Integrate multi-agent system with FastAPI backend.	Optimization — token usage, latency, error handling in agent loops. Guardrails implementation.	🤖 Claude Code Class: Debug and optimize the multi-agent project using Claude Code. GitHub Actions intro.

🔴 Phase 5: Docker, Deployment & Capstone (Weeks 21–24)

Duration: 4 weeks · 12 classes | Focus: Docker → Compose → Lovable frontend → AWS optional → Capstone Project

Outcome: Students ship a complete production-ready GenAI application with frontend, backend, and deployment

The final phase brings everything together. Docker is the primary deployment tool (mandatory). Railway or Render are used for free cloud hosting. AWS EC2 deployment is demonstrated for students who want to explore it but is not required. Lovable.dev (introduced in Level One, Week 17) is revisited for building polished frontends. The capstone combines FastAPI + LangChain + RAG + an agent + a UI + Docker.

Week	Class 1	Class 2	Class 3
Week 21	Docker — what containers are and why they matter. Dockerfile for a FastAPI + LangChain app. Build and run locally.	Docker Compose — multi-container setup: app + ChromaDB + environment variables.	Local deployment testing. docker-compose up. Secrets management. Health checks.
Week 22	Lovable.dev revisited — generate a polished frontend for a GenAI app. Streamlit vs Lovable for different use cases.	Connect Lovable frontend to FastAPI backend. CORS configuration. End-to-end testing.	🤖 Claude Code Class: Production code review. README generation. GitHub portfolio cleanup. Level 3 roadmap preview.
Week 23	Capstone Development (full class). Instructor available for unblocking. AWS EC2 optional deployment demo for interested students.	Capstone Development continued. Peer help sessions. Railway/Render free deployment.	Capstone Testing + Improvements. Bug fixes. Performance checks. UI polish.
Week 24	Final Demo Day (Part 1): Student presentations — live demo of deployed capstone application.	Final Demo Day (Part 2): Remaining presentations. Best project recognition.	🤖 Claude Code Class (Final): Advanced Claude Code features recap. Level 3 roadmap. Certificates + Course wrap-up.

Projects Summary

#	Project	Stack	Week
1	Simple GenAI Backend API	FastAPI + Groq free API	Week 5
2	AI Chatbot with Memory & Tools	FastAPI + LangChain + Memory + Tools	Weeks 11–12
3	Smart Document Assistant	RAG + MCP + Streamlit + FastAPI	Week 17
4	Multi-Agent Application	LangGraph + FastAPI	Weeks 19–20
5 (Capstone)	Full-Stack GenAI App	FastAPI + LangChain + RAG + Streamlit/Lovable + Docker	Weeks 23–24

Free Tools & APIs Used Throughout

Tool / Service	Purpose	Free Tier Details
Groq API	LLM calls in LangChain (Phase 2 onward)	Free tier — Llama 3, Mixtral models with daily limits
OpenRouter + ccr	Claude Code free usage	30+ free models. Install: npm install -g @anthropic-ai/claude-code @musistudio/claude-code-router
ChromaDB	Local vector database for RAG	Runs entirely on localhost — no account or payment needed
HuggingFace Embeddings	Document embeddings for RAG	Free open-source models via sentence-transformers library
Streamlit Cloud	Deploy Streamlit apps publicly	Free hosting for public apps at streamlit.io/cloud
Lovable.dev	AI-generated frontend UI	Free tier available — generates React apps from prompts
Railway / Render	Docker app deployment	Free tier for small apps — primary deployment platform
LangSmith	LangChain tracing and debugging	Free Developer tier — up to 5,000 traces per month

AWS EC2	Cloud deployment (optional, Week 23)	Shown for interested students — t2.micro free tier eligible

Recommended Extras & Best Practices

Buffer / Revision

- Week 1 is intentionally dense — allow students to re-read the AI hierarchy and LLM concept tables at home before Week 2
- Week 12 Class 2 is a designated buffer/catch-up class for Phase 2
- Keep Class 3 of every non-Claude-Code week flexible for Q&A, debugging, or revision — especially in Phases 2 and 3
- Add revision checkpoints at Weeks 10 and 17 before moving to the next major phase

Assignments & Practice

- Week 1 Assignment: Students write a 1-page summary in their own words — AI → ML → DL → LLM → GenAI — using analogies they invent themselves
- Assign weekly hands-on coding tasks after each class from Week 2 onward
- Mini Project 1 (Week 5): FastAPI GenAI Backend — students call a free LLM from their own API
- Mini Project 2 (Weeks 11–12): AI Chatbot — LangChain + Memory + Tools + FastAPI backend
- Mini Project 3 (Week 17): Smart Document Assistant — RAG + MCP + Streamlit + FastAPI
- Mini Project 4 (Weeks 19–20): Multi-Agent Application — LangGraph + FastAPI
- Final Capstone (Weeks 23–24): Complete production-ready GenAI application combining all phases

Week 1 Teaching Tips

- Class 1 — Build the AI hierarchy table together on the whiteboard/screen, row by row. Do not show the full table at once. Ask students to guess what comes next.
- Class 2 — Use the tiktoken library live to tokenize student names and sentences. Show the hallucination examples from Google Cloud's AI Hallucinations page. Ask: why did the model say this? Then reveal: no grounding.
- Class 3 — Use sentence-transformers in a Colab notebook. Show that 'king - man + woman = queen' in embedding space. This moment makes embeddings click for students — do not skip it.
- At the end of Week 1, tell students: every topic in the next 23 weeks connects back to something covered this week. Point to the table. Make it feel like a map, not a lecture.

Connecting to Level One

Level One (what students know)	Level Two (what they build next)
Basic ML — Linear Regression, Classification (scikit-learn)	Week 1: Understand where this fits — ML is one layer under Deep Learning and LLMs
Used requests to consume APIs	Weeks 2–5: Build their own APIs with FastAPI
Made raw LLM calls via Groq/OpenAI	Phase 2: Build structured AI apps with LangChain
Learned RAG concept in 1 class	Phase 3: Build complete RAG pipelines — embeddings, ChromaDB, retrieval, grounding
Used n8n for automation workflows	Phase 4: Build code-based agent workflows with LangGraph
Used Lovable.dev to generate a frontend	Phase 5: Connect AI backends to Lovable or Streamlit UIs
Used Cursor AI and GitHub Copilot	Throughout: Use Claude Code (free via ccr + OpenRouter) as primary AI dev assistant